

Randomized Multiresolution Band-Energy Search

A Sublinear-Time Spectral Peak Finder

Notes on `multires_bandenergy.py`

June 19, 2026

Abstract

`multires_bandenergy.py` locates a dominant (or local) spectral peak in a discrete signal while reading a number of time-domain samples that does *not* grow with the signal length N . It never computes an FFT and never makes a full pass over the data. The trick is to read a small *autocorrelation sketch* from a handful of random contiguous blocks, then answer every band-energy question analytically from that sketch via a closed-form band-integral kernel. A binary (multiresolution) search over frequency bands then converges to the peak in $O(\log)$ decisions, none of which touch the signal again. This report derives the underlying mathematics, walks through each function, and explains why the cost decouples from N .

Contents

1 Problem and Idea

We are given a real signal $x[0 .. N - 1]$ sampled at rate f_s , and we want the frequency of a dominant tone (or a local spectral peak near a seed), to some target resolution Δf . The textbook route is an FFT: $O(N \log N)$ time and a full pass over the data. Here we instead want *sublinear* access — read $\ll N$ samples — and we are willing to accept an approximate, Hz-level answer.

The central idea is a divide-and-conquer search in the frequency domain. The band $[0, f_s/2]$ is split in half; we ask which half carries more energy; we recurse into the winner. After d levels the band has width $(f_s/2) 2^{-d}$, so to reach resolution Δf we need only

$$d \approx \log_2 \frac{f_s/2}{\Delta f} \tag{1}$$

yes/no decisions. The whole game is therefore: *how do we answer “which half-band has more energy?” cheaply and reliably*, without an FFT and without rereading the signal at every level?

The answer is to precompute a small **autocorrelation sketch** once, and to turn each band-energy query into a closed-form sum over that sketch.

2 Mathematical Core

2.1 Power spectral density and autocorrelation

For a wide-sense-stationary process the autocorrelation

$$R(\tau) = \mathbb{E} x[n] x[n + \tau] \approx \frac{1}{N} \sum_n x[n] x[n + \tau] \tag{2}$$

and the power spectral density (PSD) $S(f)$ are a Fourier-transform pair (Wiener–Khinchin). With τ measured in samples and f in Hz,

$$S(f) = \sum_{\tau=-\infty}^{\infty} R(\tau) e^{-i 2\pi f \tau / f_s} = R(0) + 2 \sum_{\tau \geq 1} R(\tau) \cos\left(\frac{2\pi f \tau}{f_s}\right), \quad (3)$$

where the second form uses the fact that R is real and even ($R(-\tau) = R(\tau)$), so the imaginary parts cancel and each lag contributes a cosine.

2.2 Band energy as a closed form

The energy in a band $[f_{\text{lo}}, f_{\text{hi}}]$ is the integral of the PSD over that band:

$$E[f_{\text{lo}}, f_{\text{hi}}] = \int_{f_{\text{lo}}}^{f_{\text{hi}}} S(f) df. \quad (4)$$

Substituting (??) and exchanging sum and integral,

$$E[f_{\text{lo}}, f_{\text{hi}}] = R(0) (f_{\text{hi}} - f_{\text{lo}}) + 2 \sum_{\tau \geq 1} R(\tau) g(\tau) \quad (5)$$

where the per-lag *band kernel* is

$$g(\tau) = \int_{f_{\text{lo}}}^{f_{\text{hi}}} \cos\left(\frac{2\pi f \tau}{f_s}\right) df. \quad (6)$$

This integral is elementary. For $\tau \neq 0$,

$$g(\tau) = \frac{f_s}{2\pi\tau} \left[\sin\left(\frac{2\pi f_{\text{hi}}\tau}{f_s}\right) - \sin\left(\frac{2\pi f_{\text{lo}}\tau}{f_s}\right) \right], \quad g(0) = f_{\text{hi}} - f_{\text{lo}}. \quad (7)$$

Equations (??)–(??) are the heart of the module: **once $R(\tau)$ is known for $\tau = 0 \dots M$, the energy in any band is a finite, closed-form sum — no further data access, no FFT.** In practice R is truncated at a maximum lag M , so the sum runs $\tau = 1 \dots M$; this truncation is exactly what limits the achievable frequency resolution to $\sim f_s/M$ (the kernel oscillates in f with period f_s/τ , and the fastest term we retain has period f_s/M).

2.3 Why a contiguous block — coherent gain

The reliability of each binary decision comes from a signal-processing fact, not from sheer sample count. For a tone $A \sin(2\pi f_0 t)$ in white noise of variance σ^2 , the block autocorrelation is

$$R(\tau) \approx \frac{A^2}{2} \cos\left(\frac{2\pi f_0 \tau}{f_s}\right) + \sigma^2 \delta_{\tau,0}. \quad (8)$$

The tone leaves a *clean cosine at every lag* $\tau = 1 \dots M$, while the noise contributes *only* at $\tau = 0$. When the band in (??) contains f_0 , the cosine in (??) and the kernel $g(\tau)$ line up constructively across all M lags — an $O(M)$ coherent build-up. When the band excludes f_0 , cosine and kernel beat against each other and the terms cancel. This $O(M)$ contrast between in-band and out-of-band is what makes each yes/no decision correct.

Crucially this build-up requires *phase-coherent* samples, which a contiguous block of length L supplies. Scattered point samples (random frequencies, or random time-pairs (n, n')) carry random relative phase, give no coherent build-up, and leave only variance to average down — which is expensive. This is why the module reads *contiguous blocks* rather than scattered points.

3 Code Walkthrough

The module exposes a small set of mostly *pure* functions (in keeping with the project’s preference), built around the sketch.

3.1 ArraySampler — the access layer

A thin wrapper around a NumPy array offering point reads `sampler(indices)` and contiguous block reads `sampler.block(start, length)`. The abstraction matters: for genuinely huge signals the same search code would work against an `mmap`, disk, network, or compressed store, because the search only ever asks for a few blocks.

3.2 band_energy_kernel — the kernel $g(\tau)$ (pure)

A direct, vectorized implementation of (??). It treats $\tau = 0$ as a special case ($g(0) = f_{\text{hi}} - f_{\text{lo}}$) to avoid the $1/\tau$ singularity and applies the closed form to all nonzero lags at once:

```
out[nz] = (fs / (2.0 * np.pi * tnz)) * (  
    np.sin(2.0 * np.pi * f_hi * tnz / fs)  
    - np.sin(2.0 * np.pi * f_lo * tnz / fs)  
)  
out[~nz] = f_hi - f_lo
```

3.3 autocorrelation_sketch — estimating $R(\tau)$

This is the *only* function that reads the signal. It draws `n_blocks` contiguous blocks of length `block_len` ($\approx 4M$ by default, so every lag has ample support), each starting at a uniformly random clamped position, and for each block accumulates the biased (Bartlett) lag products

$$\hat{R}_{\text{block}}(\tau) = \frac{1}{L} \sum_{n=0}^{L-1-\tau} \tilde{x}[n] \tilde{x}[n+\tau], \quad \tau = 0 \dots M, \quad (9)$$

where $\tilde{x}$ is the (optionally) mean-removed block. Note the divisor is the fixed block length L (not the per-lag count $L - \tau$): this is the *biased* estimator, which tapers the high lags and keeps the kernel sum in (??) well-behaved (the unbiased version inflates high-lag variance). The per-block estimates are then averaged over the `n_blocks` blocks to reduce variance. The total signal access is

$$\text{samples_read} = \text{n_blocks} \times \text{block_len}, \quad (10)$$

which contains no N .

3.4 band_energy_from_sketch — evaluating E (pure)

A literal transcription of (??): build the lag vector $\tau = 0 \dots M$, evaluate $g(\tau)$ once, and return $R(0)g(0) + 2 \sum_{\tau \geq 1} R(\tau)g(\tau)$. It validates $0 \leq f_{\text{lo}} < f_{\text{hi}} \leq f_s/2$. Because only *relative* band comparisons drive the search, no absolute normalization is applied.

3.5 multiresolution_band_search — global search (pure)

The divide-and-conquer search of Section 2. At each level it splits the current band $[lo, hi]$ at the midpoint, evaluates the two half-band energies from the sketch, and keeps the higher-energy half:

```
mid = 0.5 * (lo + hi)
e_left = band_energy_from_sketch(R, fs, lo, mid)
e_right = band_energy_from_sketch(R, fs, mid, hi)
if e_left >= e_right: hi = mid      # keep left
else:                lo = mid      # keep right
```

It stops when the band narrows below `min_width_hz` (default f_s/M , the sketch’s own resolution — searching finer is meaningless) or after `max_depth` levels. The number of levels matches (??), and *every level reads zero samples*. It returns the final band, its center, and optionally a full decision trace. It is reliable when there is a single dominant band.

3.6 local_peak_search — hill-climb (pure)

When a rough location is already known, a local search is cheaper than a global one (it explores only $\log(\text{radius}/\Delta f)$ levels). Starting from a seed f_0 with an initial step (default $16\times$ resolution), it probes band energy at the current center and at $\pm \text{step}$, moves toward the larger neighbor, and *halves the step* once the center is the local maximum:

```
if e_left > e_center and e_left >= e_right: center -= step      # go left
elif e_right > e_center and e_right > e_left: center += step    # go right
else:                                     step *= 0.5          # refine
```

Probe bands are clamped to $[0, f_s/2]$. It stops when the step shrinks to the resolution. This is a 1-D coordinate ascent with geometric step refinement — essentially a ternary/golden-style climb on the (sketched) PSD.

3.7 Convenience wrappers

`find_dominant_frequency_sublinear` and `find_local_peak_sublinear` tie the pieces together: choose $M \approx \lceil f_s/\Delta f \rceil$ via `_resolution_to_max_lag`, build the sketch once, run the global or local search, and report the recovered frequency together with bookkeeping (`samples_read`, `sample_fraction`). They are the user-facing entry points.

4 Why the Cost Decouples from N

From (??), the budget is `n_blocks · block_len` $\approx$ `n_blocks` $\cdot 4M$ — no N . Each factor maps to a real requirement:

- **Resolution** $\rightarrow M$. Resolving to Δf needs lags out to $M \approx f_s/\Delta f$. A depth- M auto-correlation cannot distinguish features finer than f_s/M . So samples scale *linearly in target resolution*, not in N . (N re-enters only if you demand bin-exact $\Delta f = f_s/N$, forcing $M \approx N$.)
- **Confidence** $\rightarrow$ `n_blocks`. More blocks average down the variance of $R(\tau)$. For failure probability δ over the $\sim \log_2(f_s/2/\Delta f)$ decisions (union bound), the block count grows like $\log(1/\delta)$. Still no N .

- **Contrast** $\rightarrow$ **multiplies** `n_blocks`. With gap the normalized energy separation between winning and losing half-band, reliable decisions need

$$\text{n_blocks} \cdot M = O\left(\frac{\log(\text{depth}/\delta)}{\text{gap}^2}\right). \quad (11)$$

At *fixed SNR* the gap is N -independent: as N grows, in-band tone energy ($\propto N$) and in-band noise energy ($\propto N$) grow together, so their ratio holds. This is the deep reason the budget can stay flat.

- **Search span** $\rightarrow$ **depth**. Logarithmic per (??), and free of samples because every level reuses the one cached sketch.

In one line,

$$\text{samples_read} \approx \text{n_blocks} \cdot \frac{4f_s}{\Delta f} \approx O\left(\frac{\log(1/\delta)}{\text{gap}^2} \cdot \frac{f_s}{\Delta f}\right). \quad (12)$$

The empirical sweep bears this out: a 7130 Hz tone in noise at $\Delta f = 25$ Hz, `n_blocks` = 4, reads the *same* 30,720 samples and recovers ~ 7137 Hz across $N = 10^5, 10^6, 10^7$ — a $100\times$ range of N at identical cost and accuracy.

5 The Hard Demo: Spectral Leakage

The `_main_` block constructs a deliberately adversarial signal: eight tones of differing strength, all *gated* to a short rectangular burst in the middle of a one-million-sample record. The abrupt on/off edges convolve each tone’s line with a sinc in frequency, spreading energy into *sidelobes* (spectral leakage). The dominant 1500 Hz tone leaks far enough to partly bury the weak 2300 Hz tone beside it.

The demo shows three regimes:

1. **Global search** still picks out the strongest (1500 Hz) band despite the forest of weaker peaks and the leakage between them.
2. **Local hill-climb** with a seed inside each tone’s own neighborhood recovers every peak — even the masked 2300 Hz one.
3. **Failure mode**: seeding the weak 2300 Hz tone from farther out (2000 Hz) lets the dominant tone’s sidelobe leakage drag the climb into the 1500 Hz peak instead. This honestly illustrates the method’s limit: local search follows the local energy gradient, and leakage can reshape that gradient.

6 Summary

The module reduces spectral peak finding to (i) a one-time autocorrelation sketch read from a few coherent blocks, and (ii) a closed-form band-energy evaluation (??)–(??) that drives a logarithmic-depth search reading no further samples. The work is governed by *how finely and how confidently* you want the peak located — resolution Δf , confidence δ , spectral contrast — and not by *how long* the signal is. The honest caveats are that it assumes a dominant (or seed-localizable) peak, that truncating R at lag M caps resolution at f_s/M , and that strong spectral leakage can mislead the local climb.